

TypeScript Level-2 – Complete Documentation (Java Developer Perspective)

This document consolidates **all discussions after the previous documentation**, covering **TypeScript Level-2 core concepts** required before starting **NestJS**.

The explanations are written **side-by-side with Java comparisons** to make the mental transition smooth.

1. Async / Await (Core Concept)

Java (CompletableFuture)

```
CompletableFuture<String> future =  
    CompletableFuture.supplyAsync(() -> "data");  
  
String result = future.get();
```

TypeScript

```
async function getData(): Promise<string> {  
    return "data";  
}  
  
const result = await getData();
```

Key Differences

Java	TypeScript
Thread-based	Event-loop based
Blocking possible	Non-blocking
Verbose syntax	Clean & concise

2. Promise Basics

TypeScript Promise

```
function fetchUser(): Promise<User> {  
  return new Promise((resolve, reject) => {  
    resolve({ name: 'Rohit' });  
  });  
}
```

Preferred (async/await)

```
const user = await fetchUser();
```

3. Promise Chaining

```
getUser()  
  .then(user => user.name)  
  .catch(err => console.error(err));
```

Equivalent to Java `thenApply()`.

4. Parallel Execution - `Promise.all()`

Java

```
CompletableFuture.allOf(f1, f2).join();
```

TypeScript

```
const [user, orders] = await Promise.all([  
  getUser(),  
  getOrders()  
]);
```

✓ Faster execution ✓ Common in backend APIs

5. Error Handling

Java

```
try {  
    risky();  
} catch (Exception e) {}
```

TypeScript

```
try {  
    await risky();  
} catch (err) {  
    console.error(err);  
}
```

Important Difference

- Java → checked exceptions
- TypeScript → **no checked exceptions**

NestJS handles errors using **Exception Filters**.

6. Union Types (|) – TypeScript Special

```
let id: number | string;
```

Java has no direct equivalent.

Used heavily in APIs and DTOs.

7. Optional Properties & Optional Chaining

```
interface User {  
    name: string;  
    address?: {  
        city?: string;  
    };  
}
```

```
user.address?.city;
```

Prevents null pointer errors.

8. Nullish Coalescing (??)

```
const name = user.name ?? 'Guest';
```

Only checks `null` or `undefined` (not false / 0).

9. Union vs Optional

```
let a?: number;           // number | undefined
let b: number | null;     // explicit null allowed
```

Java equivalent:

```
Integer a = null;
```

10. Type Narrowing

```
function printId(id: number | string) {
  if (typeof id === 'string') {
    console.log(id.toUpperCase());
  } else {
    console.log(id.toFixed(2));
  }
}
```

TypeScript compiler understands runtime checks.

11. `any` vs `unknown` (CRITICAL)

 `any`

```
let data: any;  
data.foo.bar();
```

 `unknown`

```
let data: unknown;  
  
if (typeof data === 'string') {  
  console.log(data.toUpperCase());  
}
```

Rule: - `any` → disables compiler - `unknown` → safe alternative

12. Generics (Java vs TypeScript)

Java

```
class Box<T> {  
  T value;  
}
```

TypeScript

```
class Box<T> {  
  value!: T;  
}
```

13. Generic Functions

```
function wrap<T>(value: T): T {  
  return value;  
}
```

Java equivalent:

```
<T> T wrap(T value) { return value; }
```

14. Generic Constraints

```
function logLength<T extends { length: number }>(value: T) {  
  console.log(value.length);  
}
```

Used heavily in repositories and validation logic.

15. DTO-Style Typing (NestJS Critical)

```
interface CreateUserDto {  
  name: string;  
  age: number;  
}
```

Defines strict API contracts.

16. Real-World Combined Example

```
async function findUser<T>(  
  id: number,  
  mapper: (data: any) => T  
) : Promise<T | null> {  
  try {  
    const data = await db.find(id);  
    return mapper(data);  
  } catch {  
    return null;  
  }  
}
```

Uses: - async/await - Promise - Generics - Union types - Error handling

17. Interview One-Liner

TypeScript Level-2 concepts like `async/await`, union types, generics, and type narrowing make it more expressive than Java, while still preserving strong typing suitable for scalable backend systems like NestJS.

18. Readiness Checklist

After this document, you should confidently understand: - Promises and async execution - Error handling differences from Java - Union & optional types - Generics and constraints - DTO typing for APIs

✓ You are now **TypeScript-ready for advanced topics and NestJS**.

Next Step

➡ **TypeScript Level-3 (Utility types, type vs interface, immutability)** ➡ OR start **NestJS with strong TS foundation**